Instituto Federal de Educação, Ciência e Tecnologia do Rio Grande do Sul — Campus Rio Grande
Tecnologia em Análise e Desenvolvimento de Sistemas
Princípios e Padrões de Arquitetura

PROJETO INTEGRADOR · ENTREGA 1

Visão do Sistema

Community Bank: plataforma de um banco comunitário
contexto, stakeholders, drivers arquiteturais, ASRs e cenários de qualidade

Francine Gonçalves Nunes Moraes
Professora: Beatriz Costa

Branch da entrega:
`github.com/FrancineGnMoraes/community-bank-api/tree/entrega-1-visao`

Rio Grande, 2026

1 Propósito deste documento

Este documento abre o Projeto Integrador. Ele descreve **qual problema o sistema resolve, para quem e com que qualidade**, antes de qualquer decisão de estrutura. As decisões de arquitetura ficam para a Entrega 2 (Proposta Arquitetural). Aqui reunimos os **drivers arquiteturais** que vão orientar essas decisões: funcionalidades principais, cenários de atributos de qualidade, restrições e preocupações (Aulas 3 e 4).

A ideia central da disciplina guia o texto: *um sistema pode funcionar e ainda ser ruim* (Aula 1). Por isso, além de dizer o que o banco comunitário faz, este documento diz **como ele precisa se comportar**, com medidas que dá para testar.

2 Contexto do problema

Bancos comunitários são serviços financeiros solidários que geram trabalho e renda e desenvolvem a economia local, principalmente em territórios de baixa renda, apoiando a economia popular e solidária. O exemplo mais conhecido no Brasil é o **Banco Palmas** (Fortaleza-CE). Hoje existem bancos comunitários em vários estados, inclusive no Espírito Santo, articulados pela **Rede Brasileira de Bancos Comunitários**.

Na prática, esses bancos atendem moradores que muitas vezes não têm conta em banco tradicional, não conseguem crédito ou moram longe de uma agência. O atendimento costuma ser presencial, feito por pessoas da própria comunidade, com controles em papel ou planilha. Isso causa três problemas que o sistema precisa resolver:

- **Confiança:** o dinheiro é da comunidade. Toda movimentação precisa ser rastreável e a prestação de contas precisa ser pública.
- **Acesso:** o público usa principalmente o celular, nem sempre com boa conexão e nem sempre com familiaridade digital.
- **Sustentabilidade:** a equipe é pequena e o orçamento é baixo. O sistema precisa ser simples de operar e de manter.

2.1 Serviços previstos no enunciado

Os serviços abaixo vêm do enunciado do Projeto Integrador, que se baseou no material da Rede Brasileira de Bancos Comunitários:

#	Serviço	Prioridade	Entrega prevista*
S1	Abertura e extrato de conta corrente	Alta	1 (esqueleto) e 2
S2	Depósito em conta corrente	Alta	2
S3	Saque avulso ou com cartão magnético	Alta	3 e 4
S4	Crédito para financiamento de empreendimentos solidários	Alta	3 e 4
S5	Crédito para consumo pessoal e familiar, sem juros	Alta	3 e 4
S6	Prestação de contas diária e semanal para toda a comunidade	Alta	4
S7	Gerentes da conta (atendimento e acompanhamento)	Média	3
S8	Pagamento de contas (água, luz, telefone etc.)	Média	4 (integração simulada)
S9	Recebimento de aposentadorias e outros valores governamentais	Média	4 (integração simulada)
S10	Cartão de crédito popular solidário	Baixa	evolução futura

* Planejamento inicial, de acordo com o calendário da disciplina: Visão (01/09), Proposta (09/10), Detalhamento (27/10), Implementação (24/11) e Apresentação final (08/12). O planejamento pode mudar conforme o design avança. Isso é esperado no ADD, que trabalha em rodadas.

3 Objetivos de negócio

Seguindo a ideia do método PALM (Aula 3), os objetivos de negócio vêm antes dos atributos de qualidade. Cada objetivo explica **por que** um atributo importa, o que ajuda a justificar as decisões arquiteturais depois.

Objetivo de negócio	Atributo de qualidade derivado	Cenário
OB1. Ampliar a inclusão financeira de moradores sem acesso a banco tradicional	Usabilidade, desempenho em rede ruim	QA-09, QA-01
OB2. Manter a confiança da comunidade no banco	Segurança, auditabilidade, confiabilidade	QA-02 a QA-05
OB3. Fortalecer a economia local com crédito solidário cujas regras são decididas pela comunidade	Modificabilidade	QA-06
OB4. Prestar contas publicamente, todo dia e toda semana	Transparência e integridade dos dados	QA-07
OB5. Operar com equipe pequena e custo de infraestrutura próximo de zero	Disponibilidade com recursos simples, testabilidade, *buildability*	QA-08, QA-10

4 Stakeholders e preocupações

Stakeholder	Interesse no sistema	Preocupação principal
Correntista (morador)	Abrir conta, consultar saldo e extrato, depositar, sacar, pagar contas e receber benefícios	Facilidade de uso no celular; dinheiro seguro
Empreendedor solidário	Pedir e acompanhar crédito para o seu negócio	Regras de crédito claras e justas

Stakeholder	Interesse no sistema	Preocupação principal
Gerente de conta / agente de crédito	Atender no balcão, acompanhar a carteira de clientes, analisar pedidos de crédito	Operação rápida e com permissões corretas
Conselho gestor da associação comunitária	Definir políticas (limites, crédito sem juros) e aprovar a prestação de contas	Mudar regras sem depender de grandes alterações no sistema
Comunidade em geral	Ver a prestação de contas diária e semanal	Transparência sem expor dados pessoais
Parceiros externos (concessionárias, órgão pagador de benefícios, rede de cartões)	Trocar informações de pagamento e recebimento	Contratos de integração estáveis
Equipe de desenvolvimento (a aluna) e **professora**	Construir, manter, avaliar e demonstrar o sistema	Código compreensível, testável e fácil de executar

5 Funcionalidades principais (casos de uso)

Funcionalidade é a capacidade do sistema de fazer o trabalho para o qual foi feito (Aula 3). Os casos de uso centrais são:

ID	Caso de uso	Ator	Serviço
UC01	Abrir conta corrente	Correntista / gerente	S1
UC02	Consultar saldo e extrato	Correntista	S1
UC03	Depositar em conta corrente	Correntista / gerente	S2
UC04	Sacar (avulso ou com cartão)	Correntista	S3
UC05	Solicitar e acompanhar crédito (solidário ou consumo)	Correntista / empreendedor	S4, S5
UC06	Analisar e aprovar crédito	Gerente / conselho	S4, S5, S7
UC07	Pagar conta de consumo	Correntista	S8
UC08	Receber benefício governamental	Órgão pagador (sistema)	S9
UC09	Gerar a prestação de contas diária e semanal	Sistema (agendado)	S6
UC10	Consultar a prestação de contas pública	Comunidade	S6

6 Diagrama de contexto

O diagrama mostra o sistema em desenvolvimento como uma caixa só, com quem fala com ele. Esse é o ponto de partida do ADD: no começo, o único elemento a refinar é “o sistema inteiro” (Aula 3, passo 3).

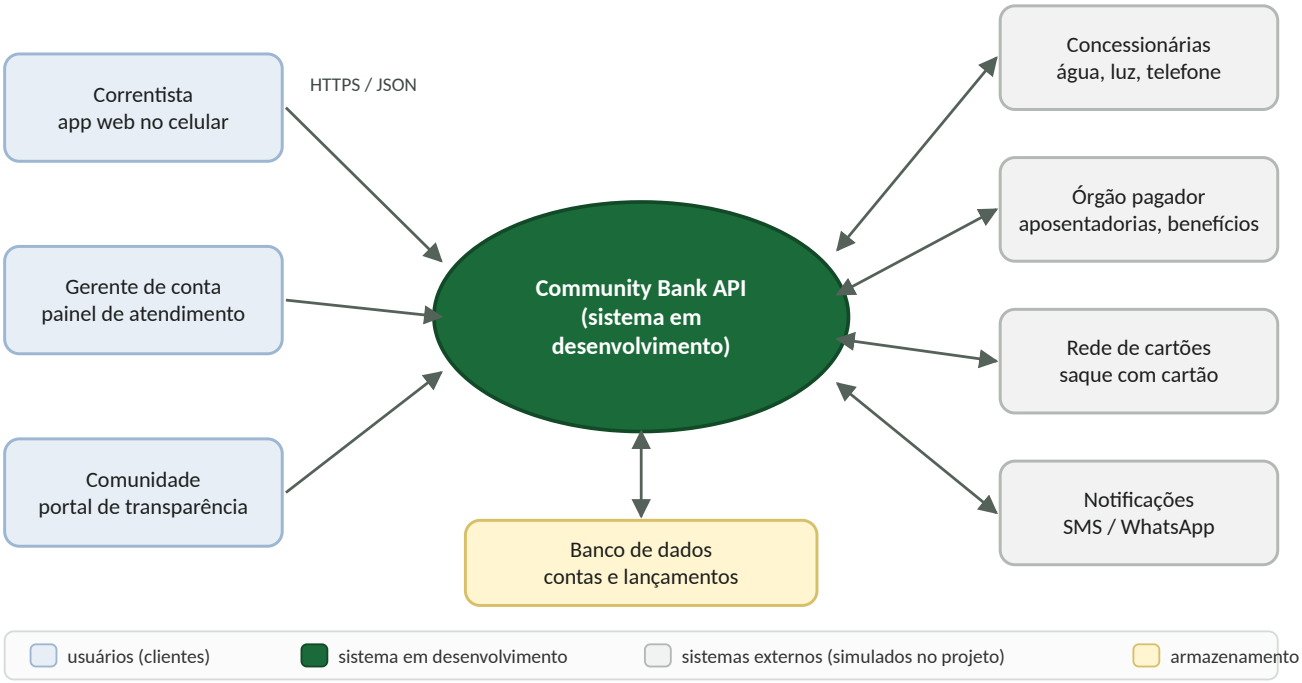


Figura 1 — Diagrama de contexto do Community Bank. Os sistemas externos serão simulados durante a disciplina.

7 Atributos de qualidade e ASRs

Dois sistemas com as mesmas funções podem ter arquiteturas completamente diferentes. O que muda a escolha são os **atributos de qualidade**. Um requisito **arquiteturalmente significativo (ASR)** é aquele que tem efeito profundo sobre a arquitetura (Aula 3). Para cada atributo relevante, transformamos o pedido vago em um requisito testável:

Como o stakeholder fala (vago)	Como o requisito fica (testável)
“O app tem que ser rápido.”	95% das consultas de saldo respondem em até 2 s com 200 usuários simultâneos.
“O sistema tem que ser seguro.”	Só usuários autenticados e autorizados acessam contas; todo acesso negado é registrado.
“A gente precisa confiar nos números.”	Toda movimentação tem autor, data, valor e origem; nenhum lançamento é alterado ou apagado.
“As regras de crédito mudam sempre.”	Uma regra nova de crédito é implementada e testada em até 1 dia, sem alterar controllers.

7.1 Cenários de atributos de qualidade (formato de seis partes)

Cada cenário segue o formato de Bass, Clements e Kazman visto em aula: **fonte, estímulo, ambiente, artefato, resposta e medida**. Ao lado do título está a prioridade (valor de negócio, risco técnico).

QA-01 · Desempenho (H, M)

Fonte	Correntista autenticado
Estímulo	Consulta o saldo ou o extrato
Ambiente	Pico: dia de pagamento de benefícios, 200 usuários simultâneos
Artefato	`GET /accounts/:id` e extrato
Resposta	Processa e devolve o saldo atualizado
Medida	95% das respostas em até 2 s

QA-02 · Segurança: autenticação (H, H)

Fonte	Usuário não autenticado
Estímulo	Tenta acessar dados de uma conta
Ambiente	Operação normal, rede aberta (internet)
Artefato	Rotas de conta e de movimentação
Resposta	Nega o acesso e registra o evento
Medida	100% das tentativas negadas, todas com registro de auditoria

QA-03 · Segurança: autorização (H, M)

Fonte	Correntista autenticado
Estímulo	Tenta ver o extrato de outra pessoa
Ambiente	Operação normal
Artefato	Caso de uso de extrato
Resposta	Recusa (403) sem revelar se a conta existe; registra o evento
Medida	0 dados de terceiros expostos; gerente vê só a própria carteira

QA-04 · Auditabilidade (H, M)

Fonte	Conselho gestor ou auditor
Estímulo	Pede a origem de um valor no saldo de uma conta
Ambiente	Operação normal, a qualquer momento
Artefato	Registro de lançamentos da conta
Resposta	Mostra os lançamentos que explicam o saldo, com autor e data
Medida	100% do saldo explicado por lançamentos; 0 lançamentos alterados ou apagados (correção = novo lançamento de estorno)

QA-05 · Confiabilidade (consistência) (H, H)

Fonte	Componente interno ou serviço externo
Estímulo	Falha no meio de uma operação (ex.: débito feito, concessionária fora do ar)
Ambiente	Operação normal com falha parcial
Artefato	Casos de uso de pagamento, saque e transferência
Resposta	Desfaz a operação inteira (transação) ou a marca como pendente
Medida	0 saldos inconsistentes; cliente informado do resultado em até 5 s

QA-06 · Modificabilidade (M, M)

Fonte	Conselho gestor
Estímulo	Aprova uma regra nova de crédito ou de limite
Ambiente	Desenvolvimento
Artefato	Política de crédito / limite no domínio
Resposta	Implementa e testa sem alterar a camada HTTP
Medida	Até 1 dia de trabalho; 0 controllers alterados

QA-07 · Transparência (H, M)

Fonte	Comunidade
Estímulo	Fechamento do dia (ou da semana)
Ambiente	Operação normal
Artefato	Relatório de prestação de contas
Resposta	Publica totais consolidados, sem dados pessoais
Medida	Publicado até 8h do dia seguinte; totais batem 100% com a soma dos lançamentos

QA-08 · Disponibilidade (M, M)

Fonte	Processo da API
Estímulo	A instância trava (crash)
Ambiente	Horário de atendimento (8h–18h)
Artefato	API em execução
Resposta	Detecta pelo `/health` e reinicia o processo
Medida	Detecção em até 30 s, volta em até 2 min; 99% no horário de atendimento

QA-09 · Usabilidade (M, M)

Fonte	Morador com pouca familiaridade digital
Estímulo	Quer consultar o saldo ou fazer um depósito
Ambiente	Celular simples, conexão 3G instável
Artefato	Cliente web + respostas da API
Resposta	Mensagens em linguagem simples, valores já formatados (R$)
Medida	Tarefa em até 3 telas; 90% de sucesso na 1ª tentativa; resposta da API < 10 KB

QA-10 · Testabilidade (M, B)

Fonte	Desenvolvedora
Estímulo	Termina uma regra de negócio
Ambiente	Desenvolvimento, antes do commit
Artefato	Domínio e casos de uso
Resposta	Testa a regra sem subir servidor nem banco
Medida	Suíte automatizada roda em menos de 10 s; toda regra tem teste

7.2 Utility tree e priorização

A utility tree organiza os ASRs: a raiz é a utilidade global do sistema, o segundo nível são os atributos, e as folhas são os cenários concretos. Cada folha recebe **valor de negócio** e **risco técnico** (H = alto, M = médio, B = baixo).

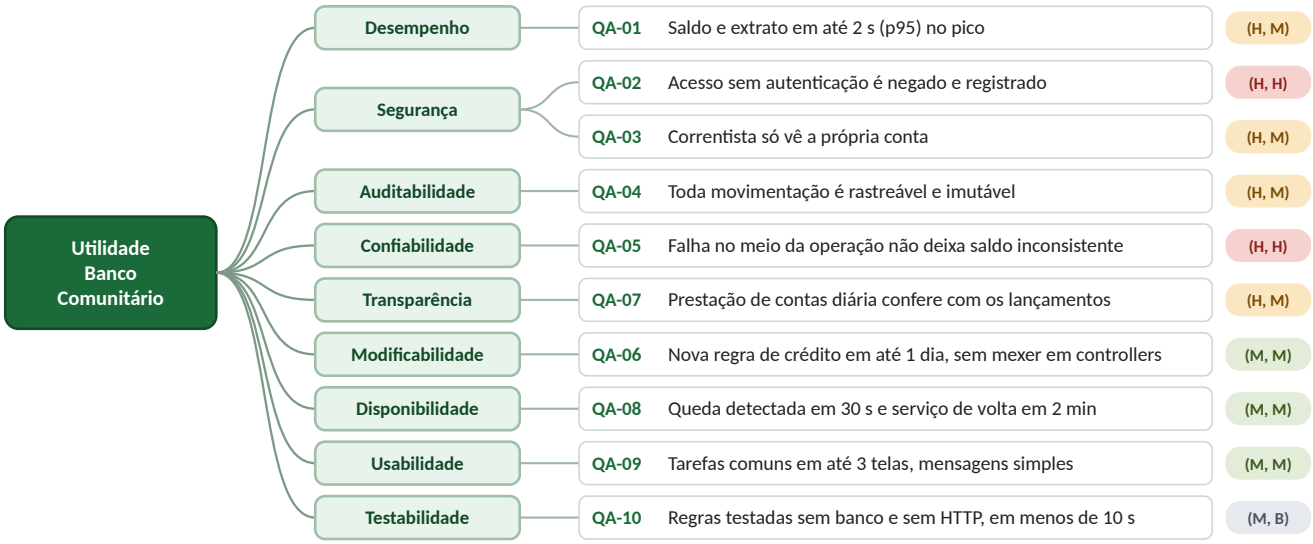


Figura 2 — Utility tree do Community Bank (valor, risco).

LEITURA DA PRIORIZAÇÃO

Os cenários **(H, H)** merecem atenção imediata: **QA-02** (autenticação) e **QA-05** (consistência em falha parcial). Ambos tratam do mesmo medo da comunidade: perder dinheiro ou ter os dados expostos. Há só dois (H, H), o que sugere que o escopo é realista para o contexto (muitos (H, H) indicariam um sistema ambicioso demais).

Os cenários (H, M) — desempenho, autorização, auditabilidade e transparência — entram logo em seguida, porque o valor é alto e já sabemos como tratá-los com táticas conhecidas.

7.3 Mapeamento para a ISO/IEC 25010 (SQuaRE)

A norma não substitui cenários e táticas, mas oferece um vocabulário comum e ajuda a não esquecer dimensões importantes (Aula 4).

Característica (ISO/IEC 25010)	Subcaracterística	Onde aparece no Community Bank
Adequação funcional	Completude, correção	Casos de uso UC01–UC10. Métrica: X = 1 − A/B (A = requisitos não implementados; B = total)
Eficiência de desempenho	Comportamento no tempo	QA-01
Compatibilidade	Interoperabilidade	Integrações com concessionárias, órgão pagador e rede de cartões
Usabilidade	Operabilidade, acessibilidade	QA-09
Confiabilidade	Disponibilidade, tolerância a falhas, recuperabilidade	QA-05, QA-08
Segurança	Confidencialidade, integridade, responsabilização, autenticidade	QA-02, QA-03, QA-04, QA-07 (sem dados pessoais)
Manutenibilidade	Modularidade, modificabilidade, testabilidade	QA-06, QA-10
Portabilidade	Instalabilidade, adaptabilidade	R4: roda em qualquer máquina com Node.js, sem serviço pago

8 Restrições

Restrições são decisões já tomadas, que o design precisa respeitar.

ID	Restrição	Origem e impacto
R1	Node.js + TypeScript + Express 5	Stack da disciplina, conforme o guia de inicialização. O tsx é usado em desenvolvimento e o tsc no build.
R2	Projeto individual no GitHub, uma branch por entrega	Enunciado do Projeto Integrador. Cada entrega precisa ser executável na sua branch.
R3	Prazos fixos	Calendário da disciplina (01/09 a 08/12). Favorece soluções simples e entregas incrementais.
R4	Custo de infraestrutura próximo de zero	Realidade de um banco comunitário. Só ferramentas livres e hospedagem simples.
R5	LGPD	O sistema guarda CPF, renda e movimentações financeiras: são dados pessoais que exigem minimização e controle de acesso.

ID	Restrição	Origem e impacto
R6	Acesso pelo celular, com conexão instável	Leva a um cliente magro (lógica no servidor) e a respostas pequenas.
R7	Sistemas externos fora do nosso controle	Concessionárias e órgão pagador serão simulados. Os contratos de integração precisam ficar isolados.

9 Preocupações arquiteturais

Preocupações (*architectural concerns*) são decisões que todo sistema precisa tomar, mesmo que nenhum stakeholder peça:

- **CRN-1. Estrutura inicial do código:** organizar módulos e camadas desde cedo, para a regra de negócio não acabar dentro das rotas.
- **CRN-2. Representação de dinheiro:** evitar erro de arredondamento de ponto flutuante (em JavaScript, 0,1 + 0,2 ≠ 0,3).
- **CRN-3. Tratamento de erros padronizado:** o mesmo formato de erro em toda a API (integridade conceitual).
- **CRN-4. Validação de entrada:** em um lugar previsível, separada das regras de negócio.
- **CRN-5. Logs e trilha de auditoria** com formato consistente.
- **CRN-6. Configuração por ambiente** (porta, banco, credenciais), fora do código.
- **CRN-7. Estratégia de testes** automatizados, por camada.
- **CRN-8. Concorrência sobre o saldo:** dois movimentos ao mesmo tempo não podem sobrescrever um ao outro (exemplo de 100 → 70 da Aula 1).

10 Backlog arquitetural inicial

O backlog reúne os drivers e as ações pendentes, organizado como Kanban (Aula 3). Ele será atualizado a cada entrega.

Não tratado	Parcialmente tratado	Completamente tratado	Descartado
QA-02 autenticação QA-03 autorização QA-05 consistência QA-07 transparência QA-08 disponibilidade CRN-5 auditoria CRN-8 concorrência	QA-01 desempenho (API leve, sem medição ainda) QA-04 auditabilidade QA-10 testabilidade CRN-6 configuração	R1 stack definida e funcionando R2 repositório e branch criados Esqueleto executável (seção 11)	Aplicativo nativo (Android/iOS): custo alto, conflita com R4 e R6

11 Esqueleto do sistema (parte prática)

A Aula 4 recomenda construir cedo uma **versão executável mínima** que valide a estrutura: infraestrutura de execução, comunicação entre componentes e uma função simples de ponta a ponta. Seguindo o guia da disciplina, a branch `entrega-1-visao` (github.com/FrancineGnMoraes/community-bank-api/tree/entrega-1-visao) contém:

```
community-bank-api/
├── src/
│   ├── app.ts        # configura o Express e
as rotas
│   └── server.ts     # abre a porta HTTP
├── package.json      # scripts
dev/build/start/typecheck
├── tsconfig.json
└── docs/Entrega1_Visao.pdf
```

Método	Rota	Objetivo
GET	`/`	confirmar que a API responde
GET	`/health`	estado básico (base para o QA-08)
GET	`/accounts`	lista de contas (dados mockados)

`app.ts` e `server.ts` ficam separados para manter a configuração da aplicação longe da decisão de abrir uma porta HTTP. Essa separação permite testar a aplicação sem subir o servidor, e a Entrega 2 já usa isso nos testes.

```
npm install
npm run dev                          # Community Bank API running on port 3000
curl http://localhost:3000/health    # {"status":"ok"}
curl http://localhost:3000/accounts # [{"id":"1","ownerName":"Maria da Silva","balance":150},
...]
```

DECISÕES QUE FICAM ABERTAS DE PROPÓSITO

Nesta entrega a rota `/accounts` devolve dados fixos, ainda sem camadas. É uma dívida técnica **intencional** (Aula 1): otimizamos para ter algo executável cedo. A organização em camadas, a persistência e a representação de dinheiro são decididas e registradas na Entrega 2.

12 Referências

- BASS, L.; CLEMENTS, P.; KAZMAN, R. *Software Architecture in Practice*. 4. ed. Addison-Wesley, 2021.
- CERVANTES, H.; KAZMAN, R. *Designing Software Architectures: A Practical Approach*. Addison-Wesley, 2016.
- CLEMENTS, P.; BASS, L. *Relating Business Goals to Architecturally Significant Requirements for Software Systems* (PALM). SEI/CMU, 2010.
- ISO/IEC 25010: *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*.
- MARTIN, R. C. *Arquitetura limpa: o guia do artesão para estrutura e design de software*. Rio de Janeiro: Alta Books, 2019.
- Material de aula da disciplina Princípios e Padrões de Arquitetura (Aulas 1, 3 e 4) e guia “Community Bank API — Node.js, TypeScript e Express”.
- Enunciado do Projeto Integrador “Criação de um Banco Comunitário”, baseado no material da Rede Brasileira de Bancos Comunitários.